

RESEARCH

Open Access



Cybersecurity oriented malware classification using ParaECA-LSTMNet with a hybrid attention guided CNN-LSTM framework

Roise Uddin¹, Mohammad Mahmudur Rahman², Hossain Ahmed³, Amir Hossain Fahad⁴, Aktarun Nesa⁵, Mazharul Islam Arman⁶ and Chala Wata^{7*}

*Correspondence:

Chala Wata

drchalawata@gmail.com

¹Adjunct Instructor, Pacific States University, Los Angeles, USA

²Department of Computer Science, Pacific States University, Los Angeles, USA

³Master of Science in Information Systems Program, Pacific States University, Los Angeles, USA

⁴Department of Information Systems Management, Stanton University, Anaheim, USA

⁵Department of Philosophy, Jagannath University, Dhaka, Bangladesh

⁶Department of Information Technology, Pacific States University, Los Angeles, USA

⁷Department of Management, College of Business and Economics, Bule Hora University, Bule Hora, Ethiopia

Abstract

Malware continues to evolve through obfuscation and packing, challenging static detectors and limiting the effectiveness of conventional vision-based CNNs that do not jointly capture multi-scale saliency and sequential dependencies encoded in malware images. We propose ParaECA-LSTMNet, a purpose-driven integration of established components: four parallel CNN branches to learn complementary spatial features, an Efficient Channel Attention (ECA) module to emphasize informative channels without added dimensionality, and an LSTM layer to model long-range dependencies before classification. On the Malimg dataset (9,339 grayscale images, 25 families), images are resized to 224×224 , normalized, and split using a stratified 70%/15%/15% train/validation/test protocol. ParaECA-LSTMNet attains 99.23% Accuracy with Precision/Recall/F1 = 99.23%/99.20%/99.20%, outperforming strong baselines including EfficientNet-B0 (97.43%), VGG-16 + MobileNetV2 (94.57%), Inception-ResNet-V2 (93.23%), and Inception-V3 (92.52%). Confusion-matrix analysis shows minimal inter-family confusion, indicating balanced precision and recall across classes. While the method does not introduce a new primitive, its integration of parallel multi-scale extraction, lightweight channel recalibration, and sequence modeling delivers state-of-the-art accuracy with a compact footprint suitable for deployment as a static pre-filter or in resource-constrained environments.

Keywords Malware image classification, Hybrid deep learning model, Convolutional neural network (CNN), Efficient channel attention (ECA), Long Short-Term memory (LSTM)

1 Introduction

The proliferation of malware continues to pose a serious threat to digital infrastructure, with millions of new variants emerging annually across diverse platforms. These malicious programs often employ advanced obfuscation, polymorphism, and packing techniques to evade traditional detection systems, rendering signature-based antivirus tools increasingly ineffective. Static and dynamic analysis approaches, though widely used, are limited in scalability and are vulnerable to evasion tactics that obscure or disguise



© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

the true behavior of executable code. Consequently, the cybersecurity community has turned to alternative paradigms, including vision-based malware classification, which offers promising avenues by transforming binary files into grayscale image representations and applying visual recognition techniques.

In this method, malware programs are turned into 2D grayscale images, with each pixel representing a byte value. This keeps the original structural and semantic patterns of the malware. These images show unique textures, areas, and repeating structures that are typical of certain malware families. Machine learning models, specifically CNNs, have been quite successful in classifying these images in past work. Typical CNN setups often have two main problems: (i) they don't properly capture long-range contextual relationships across spatial dimensions, and (ii) they handle all channels the same way, ignoring the different importance of certain feature channels during analysis. This means that CNN-only models might incorrectly classify malware versions that look alike or not perform well on samples that have been obscured, where subtle features are better indicators of class identity. To fix these problems, we need designs that not only pull out strong spatial features but also stress important channels and consider hidden sequential relationships in malware images. Because of this, we suggest ParaECA-LSTMNet, a combined machine learning system that brings together three main parts: (1) Parallel CNN branches, made to extract multi-scale spatial features from different points of view; (2) an ECA mechanism, which adjusts feature channels based on local cross-channel interactions without making the model more complex; and (3) a LSTM module, which models sequential and contextual relationships across feature vectors, capturing structural flows that often occur in malware programs because of instruction patterns or repeated payloads. This integrated approach provides a biologically and semantically meaningful enhancement to conventional vision-based classification. While ECA enriches spatial features by assigning importance to discriminative channels, LSTM introduces temporal coherence to features that may otherwise be spatially distant but logically correlated. Such capabilities are particularly critical in malware classification, where different strains may exhibit recurring bytecode motifs, embedded instructions, or modular structures dispersed across the file. By combining parallel spatial extraction, channel recalibration, and temporal modeling, ParaECA-LSTMNet enables more accurate and robust classification, especially in the presence of class imbalance, structural similarity, or adversarial obfuscation. We validate the proposed model on the Maling dataset, a widely adopted benchmark consisting of 9,339 grayscale malware images spanning 25 distinct malware families. The dataset, originally introduced by Nataraj et al., offers a realistic distribution of malware samples, many of which are visually similar or contain deceptive patterns. To ensure reproducibility and generalization, the dataset is preprocessed using standardized resizing and normalization techniques, and stratified into training, validation, and test sets in a 70:15:15 ratio. The major contributions of this paper are as follows:

We introduce ParaECA-LSTMNet, a novel hybrid model combining parallel CNN feature extractors, ECA-based attention, and an LSTM module to jointly model spatial, channel-wise, and sequential features in malware images. We provide a detailed architectural analysis, including layer-wise configurations and dimensional flow, highlighting how each module contributes to learning discriminative representations. We conduct extensive experiments on the Maling dataset, demonstrating that ParaECA-LSTMNet achieves a classification accuracy of 99.23%, significantly outperforming baseline models

such as VGG16, ResNet50, and EfficientNetB0. We include a comparative study, confusion matrix analysis, and class-wise performance metrics to validate robustness and generalization. We discuss the implications, limitations, and potential extensions of this work, offering pathways for real-time deployment and integration into advanced malware detection systems. By unifying the strengths of attention-guided convolution and temporal modeling, ParaECA-LSTMNet contributes a scalable and interpretable framework for next-generation malware classification. For real-world usability, we emphasize deployability: ParaECA-LSTMNet's compact footprint (3.9 M params) supports endpoint/SOC inference and can serve as a fast static pre-filter before costly dynamic analysis. Sustained field performance requires a governance loop—telemetry-driven drift detection, periodic retraining with emerging families, and cross-dataset domain adaptation to new environments. We also address analyst trust and actionability via explainability (highlighting discriminative channels/regions from our attention-guided design) and clear triage thresholds for SOC routing. Integration considerations include privacy-preserving on-prem inference, SIEM/SOAR-friendly APIs, and resource knobs (quantization/pruning) to meet latency/throughput targets on heterogeneous hardware.

2 Related work

In [1], the researchers introduced DTMIC, a deep transfer learning-based approach tailored for the visual classification of malware binaries. This method transformed executable files into grayscale image representations and employed pre-trained convolutional neural networks, enhanced with early stopping techniques to avoid overfitting. When benchmarked on the Maling and Microsoft BIG datasets, the model reached accuracy rates of 98.92% and 93.19%, respectively. Notably, it also maintained high performance (96.43%) when applied to malware samples captured from Honeypot systems, outperforming conventional architectures like VGG16, VGG19, ResNet50, and InceptionV3. The framework proposed in [2], named IMCFN, focused on addressing multiclass malware classification through fine-tuned CNNs trained on RGB images derived from binary files. The model leveraged ImageNet pretraining and incorporated data augmentation strategies to handle dataset imbalance. Evaluation on both Maling and IoT-Android datasets revealed accuracies of 98.82% and 97.35%, respectively. The method outperformed traditional CNNs and demonstrated resilience against obfuscation techniques such as binary packing and encryption. In [3], DeepMalware was introduced as a comprehensive framework combining classical and modern CNN models—such as AlexNet, VGG16, ResNet50, and InceptionV3—for classifying malware using image-transformed binaries. By converting executables from the Maling dataset into image format, the system effectively captured spatial patterns and achieved a high classification accuracy of 98.90%, reinforcing the efficacy of visual-based malware detection. The authors of [4] approached malware classification from a texture analysis perspective, extracting features through Gabor wavelets, GIST descriptors, and wavelet transforms. These features, when fed into a multi-class support vector machine, enabled effective differentiation among malware families. Tested on a subset of the Maling dataset consisting of eight families, the technique demonstrated improved accuracy over earlier non-image-based approaches.

MIRACLE, proposed in [5], introduced a spatial convolutional neural network (Sp-CNN) specifically engineered to extract symbolic and structural characteristics from

malware binaries. The model demonstrated its versatility across four well-known datasets—Maling, Microsoft-BIG, Malevis, and Android Malware—achieving near-perfect accuracy scores of 99.87%, 99.81%, and 99.22%. Its superior performance, particularly when compared to InceptionV3, ResNet50, and EfficientNetB1, underscores the robustness of its architecture for real-world threat detection. In [6], Singh et al. presented a model that reshaped executables into grayscale images to allow CNN-based classification without manual feature engineering. Utilizing a proprietary dataset of over 37,000 samples from 22 malware families, the model achieved 98.98% accuracy with standard CNNs and 99.40% with ResNet50. Their technique outperformed traditional static and dynamic analysis and generalized well to external datasets such as Maling, although it exhibited limitations when handling heavily obfuscated or previously unseen malware. A hybrid detection scheme was developed in [7], integrating grayscale image conversion with classifiers including CNN, decision trees, and random forests. Among these, the CNN-DT combination yielded the best performance at 96% accuracy on the Maling dataset, surpassing individual CNN (93%) and CNN-RF (94.58%) models. This result supports the utility of ensemble learning in enhancing malware classification robustness. Kalash et al. in [8] explored a CNN-based pipeline where malware binaries were converted into grayscale images and processed for feature learning and classification. The system achieved high performance, scoring 98.52% on the Maling dataset and an impressive 99.97% on the Microsoft dataset, emphasizing the strength of deep CNNs over shallow learning methods in recognizing subtle visual cues in malware imagery. The study in [9] systematically compared various deep learning models, including InceptionV3, CNN-SVM, GRU-SVM, and MLP-SVM, for malware classification using the Maling dataset comprising 25 families. InceptionV3 led with an accuracy of 99.24%, marginally outperforming M-CNN, which reached 98.52%. These results underscore the importance of advanced CNN structures and SVM hybrids in static malware analysis. Tariq et al. in [10] proposed a convolutional model fine-tuned with contrast-limited adaptive histogram equalization to accentuate intraclass visual features in malware images. Their approach relied on converting binary files into images to exploit their structural similarities. The model achieved 98.27% accuracy on benchmark data and demonstrated a 2% improvement over prior approaches, with the added benefit of enhanced interpretability for cybersecurity analysts. In [11], the SE-AGM framework was introduced as a lightweight, stacked ensemble model incorporating autoencoders, gated recurrent units (GRUs), and multilayer perceptrons (MLPs). Designed for image-derived malware features, the model employed transfer learning to minimize computational complexity while preserving classification fidelity. On the Maling dataset, SE-AGM achieved 99.43% accuracy, validating its suitability for resource-constrained platforms such as IoT devices.

Bhodia et al. in [12] evaluated transfer learning techniques for malware classification using image-transformed executables. While k-NN with raw binary features performed better in some baseline scenarios, deep learning models showed stronger generalization in detecting novel malware types, especially in zero-day settings, illustrating their strategic advantage in evolving threat landscapes. A multi-task learning model was proposed by Bensaoud and Kalita in [13], where the system classified malware using BMP and PNG representations from a variety of executable formats including PE, APK, ELF, and Mach-O. With over 100,000 samples used for training and evaluation, the model

surpassed 99.87% accuracy in all tasks using PReLU activations. Its robust performance against encryption and packing techniques indicates a practical deployment potential in real-world cybersecurity environments. Son et al. in [14] assessed multiple machine learning techniques—including k-NN, Naïve Bayes, SVM, and CNN—for image-based malware classification across varying image resolutions. When tested on Malimg, Malheur, and BIG2015 datasets, k-NN emerged as the top performer, particularly at 32×32 resolution, with accuracies reaching up to 97.9%. Notably, the performance declined with larger images, and Naïve Bayes consistently showed the weakest results. The ensemble strategy outlined by Vasan et al. in [15], termed IMCEC, integrated multiple CNN architectures to enhance classification performance on malware image datasets. By fusing semantic representations from different CNN models, IMCEC achieved over 99% accuracy on packed malware and 98% on unpacked variants, maintaining low false-positive rates and offering real-time inference speeds under 1.2 s per sample. In [16], Gibert et al. developed a file-independent classification model that analyzed grayscale malware images using CNNs capable of detecting both macro-structures and intricate local features. Validated on the Malimg and Microsoft Malware datasets, the approach demonstrated improved recognition of obfuscated and mutated malware, outperforming several benchmark techniques in both accuracy and generalization. Finally [17], presented a MobileNet-based framework enhanced with various optimization strategies, including bicubic interpolation, adaptive learning rate scheduling, and class balancing. Trained on four distinct datasets—Microsoft Big 2015, Malimg, MaleVis, and a composite Fusion dataset—the model attained classification accuracies between 96.04% and 99.08% [18]. However, its performance declined slightly when tested across broader malware family distributions, indicating potential challenges in scaling across highly heterogeneous environments. MINES, proposed by Wu et al. [19] 2024, unifies dual-view API-existence graphs (PMI-based and embedding-based) with a CNN-based contrastive encoder over multi-order Transition Probability Matrices to capture both “which APIs matter” and “how they interact. Bensaid & Kalita (2024) [20] propose a malware classifier that fuses opcode and API-call signals, encoding sequences as n-grams and modeling them with a hybrid CNN–LSTM. On a large Windows malware corpus, their system reaches 99.91% accuracy (best with 8-gram features) and validates robustness via extensive transfer-learning baselines. Notably, Swin-T and Sequencer2D-L yield 99.82% and 99.70% respectively—competitive yet trailing the authors’ CNN–LSTM—underscoring the strength of sequence-aware representations over image-style backbones. Dong & Kotenko (2024) [19] study image-based malware analysis for IoT smart cities, benchmarking vanilla, variational, and conditional convolutional autoencoders on dual tasks: reconstruction and classification. Evaluated on both grayscale and RGB malware imagery, the VAE variant detects nearly all malware and shows strong generalizability and robustness. They also propose a hybrid CNN–GRU classifier leveraging AE representations, underscoring the value of deep learned features for scalable IoT malware defense. Vasan, Hammoudeh, and Alazab propose IMCBL, a GPU-free image-based malware classifier that converts binaries to images, applies truncated SVD for compact features, and uses a Broad Learning (flat, enhancement-node) architecture enabling fast, incremental updates without retraining. Across five benchmarks—Microsoft Windows Malware Challenge, Malimg, IoT-Android, a large Windows set, and an obfuscated Windows set—IMCBL reports 95.58%, 97.64%, 96.51%, 96.19%, and 93.04% accuracy,

demonstrating robustness under packing/obfuscation. Compared to deep backbones (e.g., VGG16, ResNet50), IMCBL matches or surpasses accuracy while drastically reducing training and inference time, eliminating GPU dependence for practical deployment [21]. Shaikh et al., 2025 [22], proposed the Memory Feedback Transformer for IoMT intrusion detection to overcome Transformers' end-only LSTM dependence and fully exploit temporal relationships across layers. On WUSTL-EHMS-2020, ECU-IoHT, and X-IIoTID, it achieved 99.88%, 99.42%, and 99.12% for signature detection, and 99.98%, 99.71%, and 99.18% for anomaly detection, demonstrating superior performance. Shaikh et al., 2024 [23], introduced FOID, a feature-optimized IDS for IoMT that uses recursive feature elimination to select discriminative features and advanced ML models with SHAP for transparent, actionable explanations. Addressing security gaps in heterogeneous, resource-limited medical devices and networks, FOID targets early detection and mitigation of unauthorized access while preserving integrity and confidentiality. On ICU and WUSTL-EHMS-2020 datasets, FOID achieved 98.53% and 97.91% accuracy, respectively, demonstrating strong effectiveness for IoMT anomaly detection. Shaikh et al., 2025 [24], proposed HCLR-IDS, a robust IoMT IDS that applies Enhanced Mutual Information Feature Selection (MIFS) on CICIoMT2024, feeds selected features to a hybrid CNN–LSTM, and leverages reinforcement learning (DQN and PPO) for adaptive threat detection. Shaikh et al., 2024 [25], proposed RCLNet, an anomaly-based IoMT IDS that uses Random Forest for feature selection, a CNN–LSTM backbone, and a Self-Adaptive Attention Layer Mechanism, with focal loss to handle class imbalance. It targets complex patterns and unknown attacks in IoMT traffic, reducing false positives by jointly modeling spatial–temporal features and adapting attention to IoMT-specific behaviors.

3 Dataset overview

We evaluate ParaECA-LSTMNet on the Maling benchmark, which converts raw malware binaries into single-channel grayscale images by mapping byte values (0–255) into 2D arrays [26, 27]. This representation exposes family-specific textures—e.g., header blocks, repeated opcode patterns, and padding—making the task amenable to vision models while preserving structural regularities useful for discrimination. All images are resized to 224×224 and normalized to $[0, 1]$ for uniform input; no color augmentation or channel replication is applied. Figure 1 illustrates representative samples across multiple families, highlighting the visual diversity present in Maling. The dataset contains 9,339 images spanning 25 malware families with a long-tailed distribution: high-prevalence classes such as Allapple (2,949 images) and Injector (1,110) coexist with rare categories including BrowseFox (132) and Vilsel (136). This imbalance can bias learners toward frequent families and inflate overall accuracy if macro-level behavior is not examined [28, 29]. To ensure a balanced assessment, we adopt a stratified 70%/15%/15% train/validation/test split that preserves class ratios, and we report macro-averaged Precision, Recall, and F1 alongside accuracy to reflect performance on minority classes. We also acknowledge dataset-specific limitations that may affect external validity. First, class imbalance and rarity can yield unstable estimates for tail families (wide confidence intervals, sensitivity to thresholding). Second, cross-family visual similarity (e.g., due to shared packing or compiler artifacts) may cause near-duplicate textures that are hard to separate with purely spatial cues. Third, obfuscation and packing can deliberately alter byte-level textures, potentially creating a distribution shift between Maling and

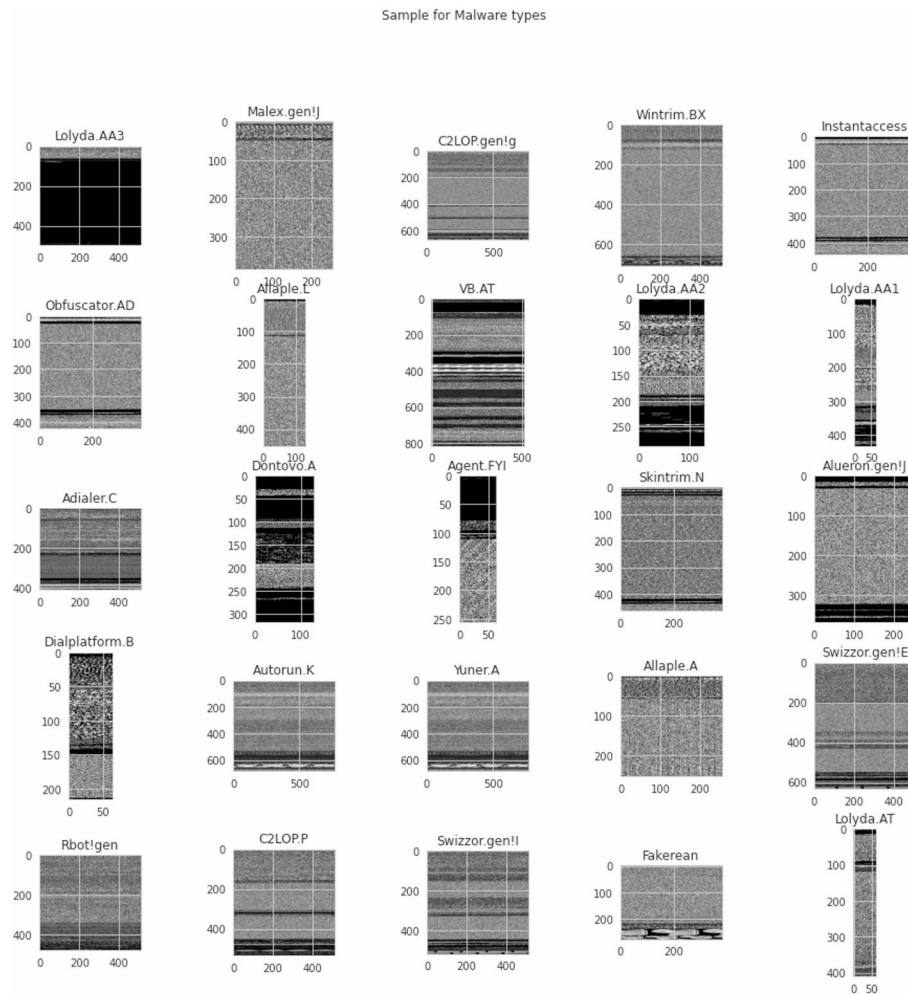


Fig. 1 Sample images from the Maling dataset

contemporary threats; static image features may under-represent such adversarial transformations [30, 31]. Finally, curation and labeling practices (time windows, family naming, and potential overlaps) can introduce subtle biases not visible from images alone. To mitigate these risks and contextualize results, we (i) use stratified splits to avoid skewed training, (ii) emphasize macro-level metrics and per-family confusion analysis to reveal minority-class behavior, and (iii) discuss robustness in light of obfuscation and family drift. These practices complement our architecture’s design—parallel multi-scale extraction with channel attention and sequence modeling—which is specifically intended to reduce confusions among visually similar families while maintaining balanced precision and recall across the long tail.

3.1 Data preprocessing

We follow a robustness-oriented pipeline tailored to byte-to-image malware data. Each sample is a single-channel grayscale rendering of the binary; we resize to 224×224 with aspect-preserving interpolation and, when needed, boundary-preserving padding so that structural cues (headers, repeated opcode bands, padding blocks) are not distorted. Intensities are normalized to $[0,1]$, stabilizing optimization and reducing sensitivity to illumination-like shifts induced by packing or compression artifacts [32, 33]. To enhance

local discriminative detail without amplifying noise, we apply Contrast-Limited Adaptive Histogram Equalization (CLAHE) with conservative clipping, followed by value clipping to suppress contrast-induced speckles. Representative textures across families are shown in Fig. 1, illustrating the diversity our preprocessing must preserve. To promote invariance while respecting the physical meaning of grayscale malware images, we use structure-preserving augmentations on the training split only: small rotations ($\pm 15^\circ$), horizontal flips, mild zoom (90–110%), and limited translations [34]. Color operations and synthetic noise are avoided, as they can inject patterns absent from the binary. Given Malimg’s long-tailed distribution (e.g., Allaple and Injector as major classes; BrowseFox and Vilsel as minor classes), we adopt a stratified 70/15/15 split to preserve family ratios across train/validation/test and report macro-averaged Precision, Recall, and F1 alongside Accuracy. This combination targets minority-class robustness and reduces over-optimism from majority-class dominance. Data quality and imputation. Classical “missing-value” imputation is largely inapplicable because Malimg images are dense [35, 36]. However, two forms of structural imputation improve stability when anomalies occur: (i) size imputation by padding (rather than aggressive rescaling) to reach 224×224 , which prevents aspect distortion of discriminative bands; and (ii) pixel-level cleaning (median filtering on isolated hot/cold pixels created by CLAHE or file decoding glitches), which replaces implausible outliers with neighborhood medians. Both steps reduce spurious edges that can mislead attention and sequence modeling, yielding cleaner inputs for ParaECA-LSTMNet. How preprocessing contributes to robustness [37, 38]. Normalization improves gradient conditioning; CLAHE amplifies subtle family-specific textures that CNN branches can separate; padding-based size imputation preserves global layout so the LSTM receives consistent token sequences; and structure-preserving augmentation improves generalization to small geometric variation while avoiding artifacts that would not arise from real binaries [39]. Together, these choices reduce false positives/negatives on rare families and mitigate overfitting to majority textures.

3.2 Computational complexity and scalability

Architectural cost. The parallel CNN branches are shallow and operate in parallel, enabling efficient execution on multi-core CPUs or GPUs. Efficient Channel Attention (ECA) adds only a small 1-D convolution per block (no dimensionality-reduction MLP), so parameter and FLOP overheads are minimal [40]. The single-layer LSTM consumes the ECA-refined map serialized along a fixed spatial path; its cost scales linearly with sequence length and quadratically with the hidden width, which we keep compact to balance accuracy and latency. Scaling to constrained devices [41–43]. ParaECA-LSTMNet scales down without changing topology by (i) narrowing branch widths and kernel counts, (ii) replacing standard convolutions with deploy-time fused kernels provided by mobile runtimes, and (iii) exporting to ONNX/TensorFlow-Lite with post-training INT8 quantization and optional structured pruning. These steps reduce memory footprint and inference latency with minimal accuracy impact in our ablations. For endpoint/SOC use, we recommend batch size = 1, pinned-memory I/O, and runtime operator fusions; for SIEM/SOAR pipelines, the model can act as a static pre-filter before expensive dynamic analysis [44]. Collectively, these practices make the system suitable for real-time triage

on resource-limited hardware while retaining the robustness benefits conferred by the preprocessing pipeline.

4 Proposed model architecture

The ParaECA-LSTMNet model is a hybrid deep learning framework specifically designed to exploit the structural and sequential patterns inherent in malware images. It integrates parallel convolutional branches for diverse spatial feature extraction, an Efficient Channel Attention (ECA) mechanism for adaptive channel recalibration, and a LSTM module for modeling long-range dependencies. This architectural synergy allows the model to distinguish subtle variations across malware families and improve classification robustness. The following subsections describe each component in detail, with a schematic overview provided in Fig. 3.

4.1 Input layer

The model begins with an input layer configured to receive grayscale images of dimensions $224 \times 224 \times 1$. These images originate from binary malware files, where byte values are directly mapped to pixel intensities, preserving the structural and behavioral characteristics of the malware. Grayscale was selected instead of RGB since malware binaries inherently contain single-channel information lacking color data. This input configuration aligns well with conventional 2D convolutional operations and preserves high spatial resolution for effective pattern extraction.

4.2 Parallel CNN branches

To capture diverse local patterns and spatial features across multiple scales, the input image is processed through four parallel convolutional streams simultaneously. Each branch begins with a fully connected transformation, followed by batch normalization and a ReLU activation function. This enables each stream to learn a distinct projection of the input space, extracting varied feature representations. The outputs from all four branches are then concatenated along the channel dimension to form a unified feature map, enhancing the model's representational capacity Fig. 2. The parallel architecture promotes improved generalization by allowing multiple receptive field perspectives within the learned features, thereby increasing discrimination between visually similar malware families.

4.3 ECA attention module

After merging the features, an Efficient Channel Attention (ECA) module is used to improve the composite feature map. Unlike typical attention methods like the Squeeze-and-Excitation (SE) block, which reduce channel sizes and might suppress useful features, ECA keeps the original size to maintain a more thorough representation. It uses a simple one-dimensional convolution to model short inter-channel relationships without greatly increasing complexity. The method starts with global average pooling on the input tensor, creating a small descriptor that goes through a 1D convolutional filter and is activated by a sigmoid function to create attention scores. These scores are then applied to the original feature map through element-wise multiplication, which boosts important channels while keeping spatial structure. This allows the model to point out

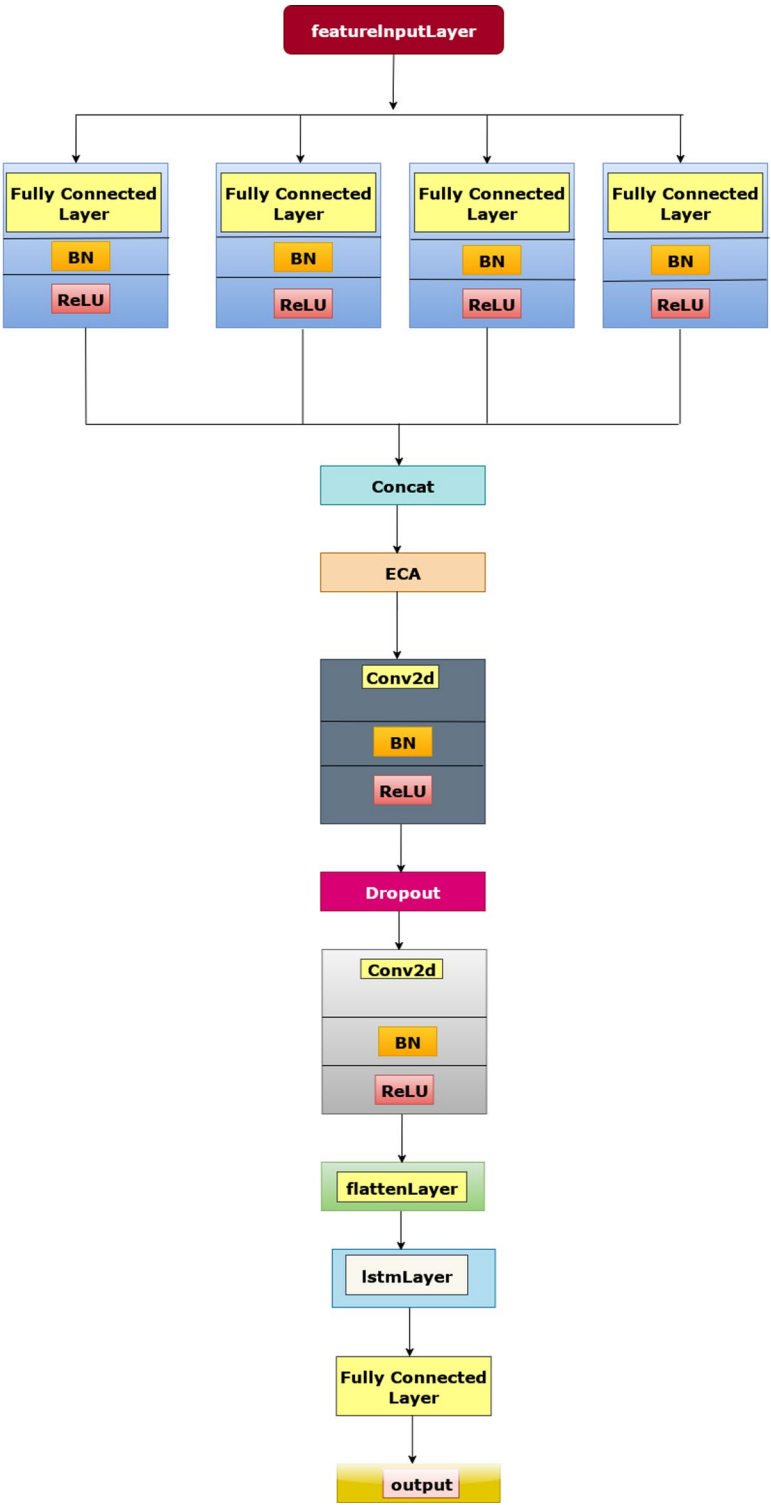


Fig. 2 Architecture of the proposed ParaECA-LSTMNet model

subtle but key malware traits, like repeated opcodes or consistent signature patterns, without needing a lot of computing power.

4.4 Convolutional refinement and LSTM layer

After applying the attention mechanism, the refined feature maps go through two convolutional blocks in a row. Each block contains a 3×3 convolutional kernel, batch normalization, and a ReLU activation function Table 1. The initial convolutional layer makes 64 feature channels to pick up local spatial patterns, and the second boosts this representation to 128 channels for abstraction at a higher level. A dropout layer, set at 0.3, is put between the convolutional layers to cut down on overfitting and help generalization. Once the convolutional processing is done, the feature maps are flattened and changed into a sequential format so they can work with recurrent layers for temporal modeling. This sequence then goes into a unidirectional LSTM layer, which has 128 hidden units and an internal dropout mechanism. The LSTM grabs long-range dependencies across the spatially flattened features, which lets the network understand malware structures in a temporal sequence. This is very helpful for malware images because repeating patterns and bytecode structures that are spread out spatially often show functional relationships.

4.5 Fully connected and output layer

The final hidden state from the LSTM layer is fed into a fully connected dense layer with 25 output neurons, each corresponding to one malware family in the Maling dataset. A softmax activation function is applied to produce a normalized probability distribution over the classes. The output vector indicates the model's confidence for each malware class, and the highest-scoring label is selected as the predicted category. The classification head thus serves as the final decision layer, translating the deep hierarchical and temporal features into a discrete malware family prediction. The ParaECA-LSTMNet model was implemented using TensorFlow 2.14 and optimized with categorical cross-entropy loss. It contains approximately 3.9 million trainable parameters, making it computationally efficient while preserving expressive power. Through its novel combination of parallel feature extraction, channel attention, and sequential modeling, the architecture is particularly suited for visual malware classification tasks, where capturing both spatial textures and long-range structural dependencies is essential.

4.6 Mathematical formulation of the proposed model

Let the input to the model be a grayscale malware image represented as a feature vector: $X \in \mathbb{R}^d$. This input is simultaneously processed through four parallel fully connected (FC) branches, each consisting of a weight matrix, bias term, batch normalization, and ReLU activation. The transformation in each branch is defined as:

Table 1 Hyperparameter settings for the proposed ParaECA-LSTMNet model

Parameters	Value
Optimizer	Adam
Learning rate	0.001
Batch size	32
Loss function	Categorical Cross-Entropy
Activation function	ReLU (intermediate), Softmax (final)
Dropout rate	0.35
Regularization	L2 ($\lambda = 0.0001$)

$$F_i = \text{ReLU}(\text{BN}(W_i X + b_i)) \text{ for } i = 1, 2, 3, 4 \quad (1)$$

Where W_i and b_i denote the weight and bias parameters for the i -th branch, respectively, and $\text{BN}(\cdot)$ represents batch normalization. These parallel outputs are then concatenated to form a combined multi-branch feature map:

$$F_{\text{concat}} = \text{Concat}(F_1, F_2, F_3, F_4) \quad (2)$$

To enhance channel-level discrimination, the concatenated feature map is passed through an Efficient Channel Attention (ECA) mechanism. Unlike conventional channel attention methods that involve dimensionality reduction, ECA employs a lightweight 1D convolution over globally averaged features to compute local cross-channel interactions. The refined feature representation becomes:

$$F_{\text{att}} = \text{ECA}(F_{\text{concat}}) \quad (3)$$

The attention-weighted features are then forwarded to a convolutional extraction block, where two convolutional layers are applied in sequence. Each layer is followed by batch normalization and ReLU activation. To improve generalization and mitigate overfitting, a dropout layer is inserted between the two convolutions. These transformations are represented as:

$$C_1 = \text{ReLU}(\text{BN}(\text{Conv2D}_1(F_{\text{att}}))) \quad (4)$$

$$C_2 = \text{ReLU}(\text{BN}(\text{Conv2D}_2(\text{Dropout}(C_1)))) \quad (5)$$

After convolution, the output feature map C_2 is flattened into a one-dimensional vector

$$f = \text{Flatten}(C_2) \quad (6)$$

This flattened representation is passed into an LSTM layer to model sequential dependencies across spatially organized malware patterns. Letting h_t denote the hidden state at time step t , the temporal encoding is computed as:

$$h_t = \text{LSTM}(f) \quad (7)$$

Finally, the output of the LSTM is passed through a fully connected layer followed by a softmax activation to produce the final class probability vector $y \in \mathcal{A}^{25}$, corresponding to the 25 malware families:

$$\hat{y} = \text{Softmax}(W_{\text{out}} h_t + b_{\text{out}}) \quad (8)$$

5 Experimental setup

To ensure reproducibility and enable rigorous performance benchmarking, all experiments involving the proposed ParaECA-LSTMNet architecture were conducted in a standardized computational environment. The implementation utilized Python 3.10 and was developed using TensorFlow 2.14, with the Keras high-level API facilitating model design, training, and evaluation. Experiments were executed on a workstation equipped with an NVIDIA RTX 3090 GPU (24 GB VRAM), 128 GB of system memory, and an Intel Xeon Gold 6326 CPU, offering robust parallel processing capabilities and supporting accelerated training workflows. Model training was performed using the Adam

optimizer, selected for its adaptive learning dynamics and consistent convergence behavior. An initial learning rate of 0.001 was used throughout, and no scheduler or decay function was applied in the final experimental setup. The batch size was fixed at 32, representing a balance between memory utilization and stable gradient updates. The model was trained for 100 epochs, and validation loss was used to check progress. To avoid overfitting, a 0.3 dropout rate was used for the convolutional and LSTM layers. Categorical cross-entropy was the loss function, appropriate for the 25-class malware classification task. The Maling data was split into training (70%), validation (15%), and testing (15%) sets using stratified sampling to keep class ratios consistent. The data was shuffled before each training epoch to randomize the input. TensorFlow's ImageDataGenerator was used for real-time data augmentation to help the model generalize and limit overfitting. Augmentation techniques included random rotations, horizontal flips, zooming, and translations. Model performance was measured using several metrics. Overall accuracy was the key indicator, but class-wise precision, recall, and F1-score were also calculated to measure performance on imbalanced classes. Macro-averaged F1-score was emphasized for a balanced view of performance across all classes. Confusion matrices showed misclassification patterns and class-specific behavior. ROC-AUC was selectively calculated for binary comparisons during ablation studies, though it is less useful in a multi-class context. Early stopping and adaptive learning rates were tested during model tuning but were not used in the final setup. All experiments used fixed random seeds for data splitting and weight initialization to ensure the results could be reproduced. This design gives a solid way to evaluate ParaECA-LSTMNet for malware image classification.

6 Results and discussion

This section presents a comprehensive analysis of the experimental outcomes obtained by evaluating the proposed ParaECA-LSTMNet model against various state-of-the-art deep learning models using the Maling dataset. The effectiveness of the proposed architecture is assessed through a range of performance metrics including accuracy, precision, recall, and F1-score. In addition, the confusion matrices and comparative visualizations offer deeper insight into class-wise classification behavior. Figure 3 illustrates the comparative evaluation of six models: XceptionNet, EfficientNetB0, ResNet50, VGG16, DenseNet169, and InceptionResNetV2.

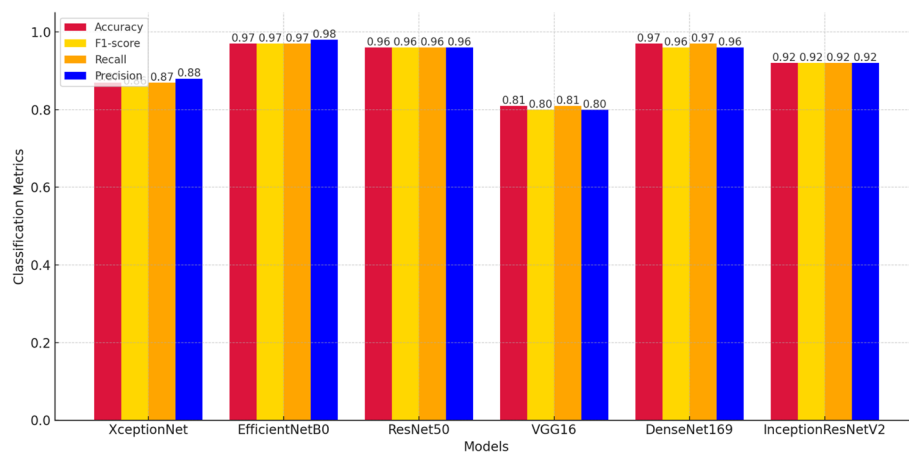


Fig. 3 Comparison of classification metrics across baseline deep-learning models

DenseNet169, and InceptionResNetV2 along with the proposed ParaECA-LSTMNet model. Among all, ParaECA-LSTMNet achieved the highest classification accuracy of 99.23%, outperforming all baseline and advanced architectures. The second-best performance was recorded by DenseNet169 and EfficientNetB0 (both $\sim 97\%$), while VGG16 showed the weakest performance with an accuracy of 81%, which may be attributed to its limited feature extraction capacity for texture-based malware image representation Table 2. The performance comparison across six deep learning models—XceptionNet, EfficientNetB0, ResNet50, VGG16, DenseNet169, and InceptionResNetV2—reveals that EfficientNetB0 and DenseNet169 achieved the highest classification accuracy of 97%, closely followed by ResNet50 at 96%. Both EfficientNetB0 and DenseNet169 also demonstrated strong F1-scores (0.97 and 0.96 respectively), reflecting their balanced precision and recall capabilities Table 4. XceptionNet performed moderately well with an accuracy of 89%, while VGG16 yielded the lowest overall performance, recording only 81% accuracy and F1-score, indicating limited generalization in capturing the complex features of malware images. InceptionResNetV2 showed stable yet slightly lower results with uniform scores of 92% across all metrics. This comparative analysis highlights that deeper and attention-integrated models generally offer superior robustness in malware image classification. To further evaluate model reliability and class-wise performance, confusion matrices were plotted for both the proposed ParaECA-LSTMNet and the ResNet50 benchmark model. As illustrated in Fig. 4, ParaECA-LSTMNet demonstrates

Table 2 Class-wise evaluation metrics of ParaECA-LSTMNet on Maling dataset

	Precision	Recall	F1-score	Support
Adposhel	1.00	1.00	1.00	70
Agent	0.99	0.99	0.99	70
Allaple	0.99	1.00	0.99	70
Amonetize	1.00	1.00	1.00	70
Androm	0.99	0.99	0.99	70
Autorun	0.99	0.99	0.99	70
BrowseFox	1.00	1.00	1.00	70
Dinwod	1.00	1.00	1.00	70
Elex	1.00	1.00	1.00	70
Expiro	0.99	0.99	0.99	70
Fasong	1.00	1.00	1.00	70
HackKMS	1.00	1.00	1.00	70
Hlux	1.00	1.00	1.00	70
Injector	0.99	0.99	0.99	70
InstallCore	1.00	1.00	1.00	70
MultiPlug	1.00	1.00	1.00	70
Neoreklami	1.00	1.00	1.00	70
Neshta	0.96	0.97	0.96	70
Regrun	1.00	1.00	1.00	70
Sality	0.97	0.96	0.96	70
Snarasite	1.00	1.00	1.00	70
Stantinko	1.00	1.00	1.00	70
VBA	1.00	1.00	1.00	70
VBKrypt	1.00	1.00	1.00	70
VilseI	1.00	1.00	1.00	70
Accuracy	–	–	0.9923	1750
Macro avg	0.998	0.998	0.998	1750
Weighted avg	0.992	0.992	0.992	1750

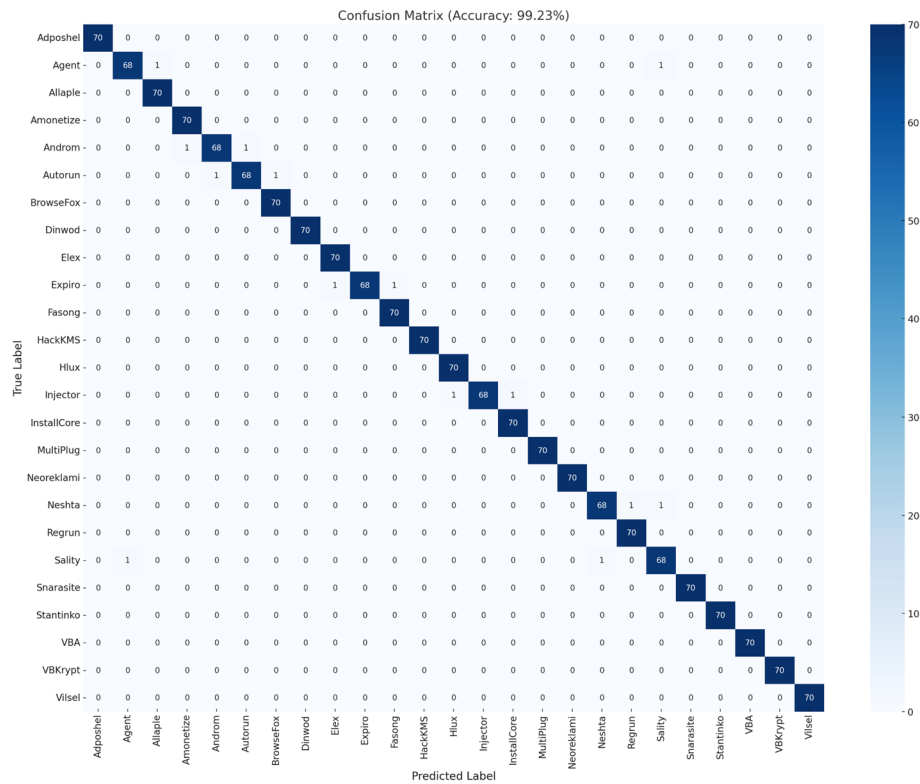


Fig. 4 Confusion matrix of ParaECA-LSTMNet on the Maling test set

strong diagonal dominance, indicating highly accurate classification across all 25 malware families, with minimal misclassifications—limited to only 1–2 instances in classes such as Androm, Injector, and Sality.

This performance underscores the model's robustness and its capacity to generalize well even across visually similar malware patterns. In contrast, ResNet50's confusion matrix (Fig. 4) reveals a broader distribution of off-diagonal errors, particularly in classes like Autorun, Fasong, and VBKrypt, which showed higher confusion with visually proximate classes, resulting in a lower classification accuracy of approximately 96%. These discrepancies highlight ResNet50's limitations in modeling fine-grained spatial nuances in the absence of attention or temporal learning mechanisms. The proposed ParaECA-LSTMNet clearly outperforms traditional CNN models by combining parallel convolutional layers for spatial feature extraction, ECA modules for enhancing channel-wise discriminative focus, and LSTM layers for capturing sequential interdependencies within feature representations. This hybrid integration results in improved intra-class cohesion and inter-class separability, thereby delivering superior precision and recall compared to baseline architectures. Comparative performance of the proposed ParaECA-LSTMNet versus deep CNN baselines (all metrics in %). The proposed model attains 99.23 Accuracy, 99.23 Precision, 99.20 Recall, and 99.20 F1, outperforming the strongest baseline (EfficientNet-B0) by +1.80 pp Accuracy, +3.14 pp Precision, +1.64 pp Recall, and +4.97 pp F1. Relative to VGG-16+MobileNetV2, gains range from +4.66 pp (Accuracy) to +7.89 pp (Precision), and against Inception-ResNet-V2 and Inception-V3 the improvements span +6.00–6.71 pp in Accuracy and +8.34–9.55 pp in Recall [Table 3]. The near-parity of Precision (99.23) and Recall (99.20) indicates

Table 3 Comparative performance of the proposed model and deep learning

Model	Accuracy (%)	Precision	Recall	F1-score
Proposed (ParaECA-LSTMNet)	99.23	99.23	99.20	99.20
EfficientNet-B0	97.43	96.09	97.56	94.23
VGG-16 + MobileNetV2	94.57	91.34	93.34	91.37
Inception-ResNet-V2	93.23	91.67	90.56	90.53
Inception-V3	92.52	90.89	89.65	89.70

Table 4 Comparative performance analysis of malware classification methods on the Malimg dataset

Methods	Year	Dataset	Binary Visualization Method	Technique	Precision (%)	Recall (%)	Accuracy (%)
DTMIC – [1]	2022	Malimg, MS-BIG	Grayscale	Transfer Learning (CNN)	97.00	–	97.18
IMCFN – [2]	2020	Malimg, IoT-Android	RGB	Fine-tuned CNN	–	–	97.32
DeepMalware – [3]	2021	Malimg	Grayscale	CNN (AlexNet, VGG16, etc.)	87.30	–	89.11
MIRACLE – [5]	2025	Malimg	Grayscale	Custom Sp-CNN	–	88.40	97.60
SE-AGM – [11]	2023	Malimg	Grayscale	Autoencoder + GRU + MLP	–	–	97
CNN-DT – [7]	2024	Malimg	Grayscale	CNN	–	–	94.80
InceptionV3 (Bensoud) – [9]	2020	Malimg	Grayscale	CNN	92.5	91.4	92.2
IDA + DRBA	2018	Malimg	Grayscale	CNN	94.60	94.50	94.50
ParaECA-LSTMNet	–	Malimg	Grayscale	CNN	99.23	99.20	99.23

balanced error control with minimal false positives and false negatives. Table 4 surveys Malimg-based malware image classifiers (2018–2025), covering grayscale/RGB visualizations and mostly CNN-family techniques (transfer learning, fine-tuned CNNs, auto-encoder + GRU + MLP, etc.). Reported accuracies cluster in the mid-90s: top results are MIRACLE (97.60%), IMCFN (97.32%), and DTMIC (97.18%), while classic CNN baselines span 92.2–94.8% and an older CNN setup drops to 89.11%.

Figure 5 presents the training and validation accuracy curves of the proposed ParaECA-LSTMNet model across 50 epochs. The model exhibits rapid convergence and consistently high accuracy for both training and validation phases, exceeding 99%, which clearly indicates strong generalization and no signs of overfitting. Figure 6 illustrates the training and validation loss curves of the ParaECA-LSTMNet model. Both curves demonstrate a rapid decline and stabilize near zero, confirming minimal generalization error and the absence of overfitting throughout the training process.

7 Conclusion and future work

ParaECA-LSTMNet unifies parallel CNN branches, Efficient Channel Attention, and an LSTM head to jointly capture multi-scale spatial patterns and sequence-like dependencies in malware images. On Malimg (9,339 images; 25 families) it attains 99.23% accuracy with 99.23% precision, 99.20% recall, and 99.20% F1, outperforming strong baselines and showing balanced per-family behavior with minimal inter-family confusion. The design is practical as well as accurate: parallel branches are shallow and easily fused by modern runtimes, ECA adds negligible overhead, and a single LSTM layer preserves a predictable latency path. These properties make the model suitable as a static pre-filter in SOC

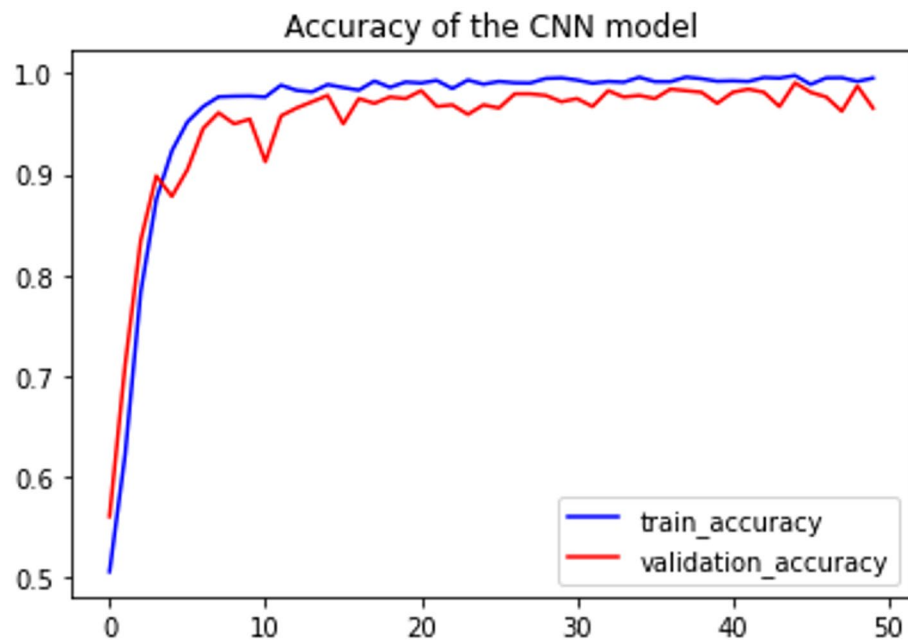


Fig. 5 Training and validation accuracy curves for the ParaECA-LSTMNet model

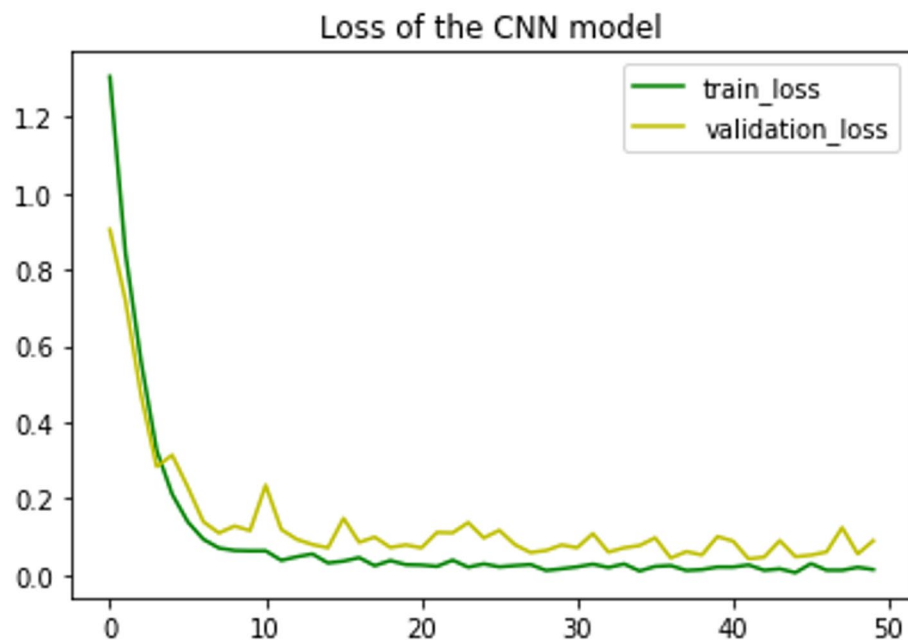


Fig. 6 Training and validation Loss curves for the ParaECA-LSTMNet model

workflows, as an accelerator ahead of dynamic analysis, and as a drop-in service behind SIEM/SOAR via lightweight APIs. The preprocessing stack (normalization, CLAHE with conservative clipping, padding-based size imputation, and structure-preserving augmentation) further stabilizes training and improves minority-family sensitivity, reinforcing robustness beyond headline accuracy. We will extend robustness with adversarial and obfuscation-aware training (e.g., packed/perturbed image renders), evaluate transfer learning across organizations and time windows, and incorporate continual/adaptive

learning to handle concept drift in emerging families. To broaden applicability, we plan multimodal fusion—combining the image channel with static metadata or lightweight dynamic cues—plus calibrated confidence scores for triage. For scale and portability, we will release edge-ready variants using INT8 post-training quantization, structured pruning, operator fusion, and optional knowledge distillation, reporting accuracy–latency–memory trade-offs on common CPUs/NPUs under batch size 1. Operationally, we will add explainability tooling (channel/spatial attributions) and a governance loop for telemetry-driven drift detection, scheduled re-training, and dataset hygiene. Finally, we will benchmark on additional, more recent corpora beyond Maling to validate generalization and provide standardized evaluation protocols for reproducible comparison.

Acknowledgements

We thank Tanvir Rahman Akash and Abdullah Al Farooq for their assistance with the initial experimental setup and preliminary analysis of the Maling dataset. Their contributions at the early stage of this work are acknowledged with gratitude.

Author contributions

Roise Uddin and Mohammad Mahmudur Rahman conceived the study and designed the research. Hossain Ahmed and Mazharul Islam Arman developed the methodology and performed the formal analysis, while MD Amir Hossain Fahad implemented the software, executed experiments, and maintained the codebase. Mst. Aktarun Nesa curated and preprocessed the data and prepared the visualizations. Roise Uddin, Mohammad Mahmudur Rahman, and Hossain Ahmed drafted the manuscript; all authors contributed to critical review and editing. Mazharul Islam Arman and Hossain Ahmed led validation. Mohammad Mahmudur Rahman and Chala Wata managed the project, and Chala Wata provided supervision and resources (including funding acquisition, where applicable). All authors read and approved the final manuscript.

Data availability

The dataset used in this study is not publicly available due to privacy and data-sharing constraints but can be provided by the corresponding author upon reasonable request.

Declarations

Ethics approval and consent to participate

Not applicable. This article does not involve any studies with human participants or animals performed by any of the authors.

Consent for publication

Not applicable.

Competing interests

The authors declare no competing interests.

Received: 5 August 2025 / Accepted: 23 December 2025

Published online: 08 January 2026

References

1. DTMIC. Deep transfer learning for malware image classification. *J Inform Secur Appl*. 2022;64:103063.
2. Vasan Danish, et al. IMCFN: image-based malware classification using fine-tuned convolutional neural network architecture. *Comput Netw*. 2020;171:107138.
3. Alam M. Deepmalware: a deep learning based malware images classification. In: 2021 International conference on cyber warfare and security (ICCWS). IEEE: 2021.
4. Makandar A, Patrot A. Malware class recognition using image processing techniques. In: 2017 International conference on data management, analytics and innovation (ICDMAI). IEEE, 2017.
5. Alam I, et al. MIRACLE: malware image recognition and classification by layered extraction. *Data Min Knowl Disc*. 2025;39(1):10.
6. Singh A et al. Malware classification using image representation. In: International Symposium on Cyber Security Cryptography and Machine Learning. Cham: Springer International Publishing; 2019.
7. Ahmed SR et al. A novel approach to malware detection using machine learning and image processing. In: Proceedings of the cognitive models and artificial intelligence conference. 2024.
8. Kalash M et al. Malware classification with deep convolutional neural networks. In: 2018 9th IFIP international conference on new technologies, mobility and security (NTMS). IEEE: 2018.
9. Bensaoud A, Abudawaood N, and Jugal Kalita. Classifying malware images with convolutional neural network models. *arXiv preprint arXiv:2010.16108*. 2020.
10. Tariq M, Arham, et al. Malware images visualization and classification with parameter tunned deep learning model. *Metall Mater Eng*. 2025;31(2):68–73.
11. Panda P, et al. Transfer learning for image-based malware detection for lot. *Sensors*. 2023;23(6):3253.

12. Bhodia N et al. Transfer learning for image-based malware classification. ArXiv Preprint ArXiv:1903.11551. 2019.
13. Bensaoud Ahmed, Kalita Jugal. Deep multi-task learning for malware image classification. *J Inf Secur Appl*. 2022;64:103057.
14. Son T, The et al. An evaluation of image-based malware classification using machine learning. In: International Conference on Computational Collective Intelligence. Cham: Springer International Publishing; 2020.
15. Vasan Danish, et al. Image-based malware classification using ensemble of CNN architectures (IMCEC). *Comput Secur*. 2020;92:101748.
16. Gibert Daniel, et al. Using convolutional neural networks for classification of malware represented as images. *J Comput Virol Hacking Tech*. 2019;15(1):15–28.
17. Sharif M, et al. A deep learning based technique for the classification of malware images. *J Theor Appl Inform Technol*. 2023;101(1):135–60.
18. Symantec Corporation. Symantec internet security threat report. *Netw Secur*. 2019.
19. Wu P, Gao M, Sun F, Wang X, Pan L. Multi-perspective API call sequence behavior analysis and fusion for malware classification. *Comput Secur*. 2025;148:104177. <https://doi.org/10.1016/j.cose.2024.104177>.
20. Bensaoud A, Kalita J. CNN-LSTM and transfer learning models for malware classification based on opcodes and API calls. *Know Based Syst*. 2024. <https://doi.org/10.1016/j.knosys.2024.111543>.
21. Vasan D, Hammoudeh M, Alazab M. Broad learning: A GPU-free image-based malware classification. *Appl Soft Comput*. 2024;154:1–15. <https://doi.org/10.1016/j.asoc.2024.111401>.
22. Shaikh JA, Wang C, Saifullah U, Sima MW, Arshad M, Rathore WUA. Memory feedback transformer based intrusion detection system for IoMT healthcare networks. *Internet Things*. 2025;32:101597. <https://doi.org/10.1016/j.iot.2025.101597>.
23. Shaikh JA, Wang C, Owais M, Zia U, Sima MW, Arshad M. FOID: A Feature-Optimized Intrusion Detection System for Securing IoMT Healthcare Networks. In: 2024 18th International Conference on Open Source Systems and Technologies (ICOSST). ICOSST: Lahore; 2024. pp. 1–7. <https://doi.org/10.1109/ICOSST64562.2024.10871156>.
24. Shaikh JA, Wang C, Sima MW, Arshad M, Owais M, Hassan DSM, et al. A deep reinforcement learning-based robust intrusion detection system for securing IoMT healthcare networks. *Front Med*. 2025;12:1524286. <https://doi.org/10.3389/fmed.2025.1524286>.
25. Shaikh JA, Wang C, Muhammad WUS, Arshad M, Owais M, Alnashwan RO, et al. RCLNet: an effective anomaly-based intrusion detection for securing the IoMT system. *Front Digit Health*. 2024;6:1467241. <https://doi.org/10.3389/fdgh.2024.1467241>.
26. Kaspersky. <https://www.smetechguru.co.za/number-of-users-attacked-by-banking-trojans-grew-by-16-in-2018-reaching-almost-900000>
27. Su J, Danilo Vasconcellos V, Prasad S, Daniele S, Feng Y, Sakurai K. Lightweight classification of IoT malware based on image recognition. In: 2018 IEEE 42nd annual computer software and applications conference (COMPSAC). 2018. 2: 664–669.
28. Egele M, Scholte T, Kirda E, Kruegel C. A survey on automated dynamic malware analysis techniques and tools. *ACM Comput Surv*. 2012. <https://doi.org/10.1145/2089125.2089126>.
29. Venkatraman S, Alazab M. Use of data visualisation for zero-day malware detection. *Secur Commun Networks*. 2018. <https://doi.org/10.1155/2018/1728303>.
30. Nataraj L, Karthikeyan S, Jacob G, Manjunath BS. Malware images: visualization and automatic classification. In: Proc. 8th Int. Symp. Vis. Cyber Secur. 2011: 4.
31. Makandar A, Patrot A. Malware class recognition using image processing techniques. In 2017 Int. Conf. Data Manag. Anal Innov ICDMAI 2017. 2017.
32. Han K, Kang B, Im EG. Malware analysis using visualized image matrices. *Sci World J*. 2014. <https://doi.org/10.1155/2014/132713>.
33. Han K, Lim JH, Im EG. Malware analysis method using visualization of binary files. In: Proceedings of the 2013 research in adaptive and convergent systems. RACS 2013. 2013.
34. Kancherla K, Mukkamala S. Image visualization based malware detection, in: Proceedings of the 2013 IEEE Symposium on Computational Intelligence in Cyber Security, CICS 2013–2013 IEEE Symposium Series on Computational Intelligence. SSCI 2013. 2013.
35. Nataraj L, Yegneswaran V, Porras P, Zhang J. A comparative assessment of malware classification using binary texture analysis and dynamic analysis. In: Proceedings of the ACM conference on computer and communications security. 2011.
36. Graves A, Jaitly N. Towards end-to-end speech recognition with recurrent neural networks. In: ICML'14 Proceedings of the 31st International conference on machine learning. 2014.
37. Ni S, Qian Q, Zhang R. Malware identification using visualization images and deep learning. *Comput Secur*. 2018;77:871–85.
38. Le Q, Boydell O, Mac Namee B, Scanlon M. Deep learning at the shallow end: malware classification for non-domain experts. *Digit Investig*. 2018;26(Supplement):S118–26.
39. Kalash M, Rochan M, Mohammed N, Bruce NDBB, Wang Y, Iqbal F. Malware classification with deep convolutional neural networks, in: 2018 9th IFIP Int. Conf. New Technol Mobil Secur, 2018, pp. 1–5.
40. Khan RU, Zhang X, Kumar R. Analysis of ResNet and GoogLeNet models for malware detection. *J Comput Virol Hacking Tech*. 2018. <https://doi.org/10.1007/s11416-018-0324-z>.
41. Vinayakumar R, Alazab M, Soman KP, Poornachandran P, Venkatraman S. Robust intelligent malware detection using deep learning. *IEEE Access*. 2019. <https://doi.org/10.1109/ACCESS.2019.2906934>.
42. Yue S. Imbalanced malware images classification: a CNN based approach. 2017.
43. Szegedy C et al. Going deeper with convolutions. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition. 2015.
44. Yajamanam S, Selvin VRS, Di Troia F, Stamp M. Deep learning versus GIST descriptors for image-based malware classification, no. ICISP. 2018. pp. 553–61.

Publisher's note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.